

# ЛЕКЦИЯ 1. Архитектура современных высокопроизводительных вычислительных систем и основы параллелизма

## Цели лекции

По завершении лекции слушатель должен:

1. Сформировать понимание архитектурных особенностей многоядерных процессоров с общей памятью (SMP-систем).
2. Знать и уметь применять фундаментальные законы параллельных вычислений — закон Амдала и закон Густафсона.
3. Разбираться в иерархии памяти современных процессоров и оценивать её влияние на производительность вычислительных алгоритмов.
4. Освоить Roofline-модель как аналитический инструмент для определения узких мест вычислительных алгоритмов.
5. Понимать механизмы когерентности кэшей (протокол MESI) и уметь выявлять проблему ложного совместного использования (false sharing).
6. Владеть основами работы с NUMA-архитектурами и привязкой потоков к ядрам процессора.

## 1. ВВЕДЕНИЕ: МОТИВАЦИЯ, КЛАССИФИКАЦИЯ ФЛИННА, ЗАКОНЫ АМДАЛА И ГУСТАФСОНА

### 1.1. Актуальность проблемы

Современные вычислительные системы переживают фундаментальный сдвиг парадигмы. В период с 1970 по 2005 год производительность процессоров росла преимущественно за счёт увеличения тактовой частоты — с 1 МГц до 4 ГГц. Однако после 2004–2005 годов дальнейшее увеличение частоты стало невозможным из-за физических ограничений: экспоненциального роста тепловыделения, токов утечки и ограничения скорости распространения сигнала в кристалле.

Этот феномен получил название «стена частоты» (**frequency wall**) и «стена питания» (**power wall**). Начиная с 2005 года, развитие процессорных архитектур пошло по пути увеличения количества ядер на кристалле: от 2 ядер (Intel Core Duo, 2006) до 128 и более ядер в современных серверных процессорах (AMD EPYC Genoa, 2023). Параллельно развиваются гетерогенные архитектуры, сочетающие центральные процессоры с графическими ускорителями (GPU), тензорными процессорами (TPU) и специализированными ускорителями.

Следствием этого сдвига является то, что **последовательные программы не способны использовать потенциал современного оборудования**. Программист, не владеющий методами параллельного программирования, теряет от 90 до 99% доступной вычислительной мощности.

Дополнительным фактором является «стена памяти» (**memory wall**): производительность процессоров растёт значительно быстрее, чем пропускная способность оперативной памяти. Разрыв между пиковой вычислительной производительностью процессора и пропускной способностью памяти достигает 100–200 раз для операций с плавающей точкой одинарной точности.

*Научное обоснование:* В классической работе Wulf, W. A., & McKee, S. A. (1995). *Hitting the memory wall: Implications of the obvious*. ACM SIGARCH Computer Architecture News, 23(1), 21–24 было впервые формализовано расхождение между скоростью процессоров и памяти, ставшее основным драйвером развития многоуровневой иерархии памяти.

## 1.2. Классификация Флинна

В 1966 году Майкл Флинн предложил классификацию вычислительных архитектур по количеству потоков команд и потоков данных:

| Класс                                             | Поток команд | Поток данных | Примеры                                                 |
|---------------------------------------------------|--------------|--------------|---------------------------------------------------------|
| <b>SISD</b> (Single Instruction, Single Data)     | Один         | Один         | Классические последовательные процессоры                |
| <b>SIMD</b> (Single Instruction, Multiple Data)   | Один         | Множество    | Векторные расширения (SSE, AVX, ARM NEON), GPU          |
| <b>MISD</b> (Multiple Instruction, Single Data)   | Множество    | Один         | Практически не используется (теоретические архитектуры) |
| <b>MIMD</b> (Multiple Instruction, Multiple Data) | Множество    | Множество    | Многоядерные процессоры, кластеры, суперкомпьютеры      |

Для данной дисциплины ключевыми являются:

- **SIMD** — векторизация вычислений внутри одного ядра (AVX-512 обрабатывает 16 чисел float за одну инструкцию).
- **MIMD** — многопоточное программирование на многоядерных процессорах с общей памятью (тема данного семестра).

*Ссылка:* Flynn, M. J. (1966). *Very High-Speed Computing Systems*. Proceedings of the IEEE, 54(12), 1901–1909.

## 1.3. Закон Амдала

Закон Амдала (1967) определяет теоретическое максимальное ускорение программы при параллелизации её части. Если доля параллелизуемого кода составляет  $P$ , а количество процессоров равно  $N$ , то ускорение  $S$  ограничено:

$$S = \frac{1}{(1 - P) + \frac{P}{N}}$$

где: -  $S$  — ускорение (speedup), -  $P$  — доля параллелизуемого кода ( $0 \leq P \leq 1$ ), -  $N$  — количество процессоров (ядер).

**Ключевые следствия:** - При  $P = 0.9$  (90% кода параллелизуемо) и  $N \rightarrow \infty$ :  $S_{max} = 10$ . - При  $P = 0.99$  и  $N \rightarrow \infty$ :  $S_{max} = 100$ . - Последовательная часть кода  $(1 - P)$  является **фундаментальным ограничением**.

*Ссылка: Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. Proceedings of AFIPS '67, 483–485.*

## 1.4. Закон Густафсона

Закон Густафсона (1988) предлагает иной взгляд на проблему: при увеличении числа процессоров можно увеличивать и объём решаемой задачи, сохраняя время решения постоянным. Ускорение по Густафсону:

$$S = N - P(N - 1) = 1 + (1 - P)(N - 1)$$

где  $P$  — доля последовательного кода (в отличие от закона Амдала, где  $P$  — доля параллельного кода).

**Ключевое отличие:** Закон Густафсона оптимистичнее закона Амдала, поскольку предполагает, что с ростом вычислительных ресурсов мы решаем **более крупные задачи**, а не ускоряем фиксированную.

*Ссылка: Gustafson, J. L. (1988). Reevaluating Amdahl's Law. Communications of the ACM, 31(5), 532–533.*

## 1.5. Пример расчёта ускорения

Для иллюстрации различий двух законов рассмотрим пример: задача обработки спутникового изображения размером  $10000 \times 10000$  пикселей. Последовательная часть (загрузка данных, инициализация) занимает 5% времени. Параллельная часть (фильтрация, преобразование) — 95%.

Расчёт для  $N = 16$  ядер: - Амдал:  $S = 1 / (0.05 + 0.95 / 16) = 1 / 0.109 \approx 9.17$  - Густафсон:  $S = 16 - 0.05 \cdot 15 = 15.25$

Пример расчёта:

*// Фрагмент 1: Расчёт ускорения по законам Амдала и Густафсона*

```
#include <iostream>
#include <iomanip>
#include <vector>
```

```
// Закон Амдала: S = 1 / ((1 - P) + P/N)
// P - доля параллелизуемого кода, N - количество ядер
double amdahls_law(double P, int N) {
    return 1.0 / ((1.0 - P) + P / static_cast<double>(N));
}
```

*// Закон Густафсона:  $S = N - p \cdot (N - 1)$ , где  $p$  - доля последовательного кода*  
*//  $p = 1 - P$*

```
double gustafsons_law(double P, int N) {  
    double p = 1.0 - P; // доля последовательного кода  
    return N - p * (N - 1);  
}
```

```
int main() {  
    const double P = 0.95; // 95% кода параллелизуемо  
    const std::vector<int> cores = {1, 2, 4, 8, 16, 32, 64, 128};  
  
    std::cout << std::fixed << std::setprecision(2);  
    std::cout << "Доля параллельного кода: " << P * 100 << "%\n\n";  
    std::cout << std::left << std::setw(10) << "Ядра"  
        << std::setw(15) << "Амдал"  
        << std::setw(15) << "Густафсон" << "\n";  
    std::cout << std::string(40, '-') << "\n";
```

```
    for (int N : cores) {  
        double s_amdahl = amdahls_law(P, N);  
        double s_gustafson = gustafsons_law(P, N);  
        std::cout << std::left << std::setw(10) << N  
            << std::setw(15) << s_amdahl  
            << std::setw(15) << s_gustafson << "\n";  
    }
```

*// Предельное ускорение по Амдалу при  $N \rightarrow$  бесконечность*  
double s\_max\_amdahl = 1.0 / (1.0 - P);  
std::cout << "\nПредельное ускорение (Амдал,  $N \rightarrow \infty$ ): "  
 << s\_max\_amdahl << "\n";

```
    return 0;  
}
```

## 2. ИЕРАРХИЯ ПАМЯТИ: РЕГИСТРЫ, L1/L2/L3, RAM

### 2.1. Общая структура иерархии памяти

Современный процессор использует многоуровневую иерархию памяти для минимизации среднего времени доступа к данным. Принцип основан на **локальности** (пространственной и временной): данные, к которым обращались недавно, с высокой вероятностью будут запрошены снова; данные, расположенные рядом в памяти, с высокой вероятностью будут запрошены в ближайшее время.

| Уровень                         | Объём (типично)  | Задержка (циклы) | Задержка (нс) | Пропускная способность |
|---------------------------------|------------------|------------------|---------------|------------------------|
| Регистры                        | ~1 КБ            | 1                | ~0.3          | ~1000+ ГБ/с            |
| L1 кэш<br>(данные)              | 32–64 КБ         | 4–5              | ~1.5          | ~100 ГБ/с              |
| L1 кэш<br>(инструкции)          | 32–64 КБ         | 4–5              | ~1.5          | ~100 ГБ/с              |
| L2 кэш                          | 256 КБ – 1.25 МБ | 12–14            | ~4–5          | ~50 ГБ/с               |
| L3 кэш                          | 8–64 МБ          | 40–50            | ~12–15        | ~30 ГБ/с               |
| Оперативная<br>память<br>(DDR5) | 16–512 ГБ        | 200–300          | ~60–100       | ~50–100 ГБ/с           |
| NVMe SSD                        | 1–8 ТБ           | ~100 000         | ~10 000       | ~7 ГБ/с                |

*Примечание:* Данные приведены для процессоров семейства Intel Xeon Sapphire Rapids (2023) и AMD EPYC Genoa. Для процессоров ARM (Neoverse N2, Graviton3) характеристики аналогичны.

*Ссылка:* Hennessy, J. L., & Patterson, D. A. (2017). *Computer Architecture: A Quantitative Approach* (6th ed.). Morgan Kaufmann.

### 2.2. Кэш-линии (Cache Lines)

Память читается и записывается не побайтово, а блоками фиксированного размера — **кэш-линиями**. В большинстве современных архитектур (x86-64, ARM) размер кэш-линии составляет **64 байта**.

**Следствия:** - При обращении к одному байту процессор загружает в кэш 64 байта. - Это основа **пространственной локальности**: если программа обратилась к элементу  $A[i]$ , с высокой вероятностью в ближайшее время она обратится к  $A[i+1]$ . - Аппаратные **предвыборщики (hardware prefetchers)** используют этот паттерн для загрузки данных до того, как они будут запрошены.

### 2.3. Типы промахов кэша

Различают три типа промахов кэша (классификация 3C — Compulsory, Capacity, Conflict):

1. **Compulsory miss (обязательный):** Первый доступ к данным, которых ещё нет в кэше.
2. **Capacity miss (по ёмкости):** Данные вытеснены из кэша из-за его ограниченного размера.

3. **Conflict miss (конфликтный):** Данные вытеснены из-за ограниченной ассоциативности кэша (несколько адресов конкурируют за один набор).

## 2.4. Влияние шага доступа к памяти (stride)

Шаг доступа к памяти (stride) критически влияет на производительность. При последовательном доступе (stride = 1) аппаратный предвыборщик эффективно загружает данные. При большом шаге (stride > 64 байта) каждый доступ порождает промах кэша.

Пример: обход массива из 1 миллиона элементов float с шагом 1, 16 и 256 элементов. Разница в производительности может достигать **50–100 раз**.

Пример кода для анализа производительности при различных шагах доступа:

*// Фрагмент 2: Анализ производительности при различных шагах доступа к памяти*

```
#include <iostream>
#include <vector>
#include <chrono>
#include <iomanip>
#include <numeric>
```

*// Обход массива с заданным шагом*

```
long long traverse_with_stride(const std::vector<int>& data, int stride) {
    long long sum = 0;
    for (size_t i = 0; i < data.size(); i += stride) {
        sum += data[i];
    }
    return sum;
}
```

```
int main() {
    const size_t N = 100'000'000; // 100 миллионов элементов
    std::vector<int> data(N);
    std::iota(data.begin(), data.end(), 1); // Заполнение 1, 2, 3, ...

    const std::vector<int> strides = {1, 2, 4, 8, 16, 32, 64, 128, 256};

    std::cout << std::left << std::setw(10) << "Шаг"
              << std::setw(20) << "Время (мс)"
              << std::setw(20) << "Эффект. пропуск. (ГБ/с)" << "\n";
    std::cout << std::string(50, '-') << "\n";

    for (int stride : strides) {
        // Разогрев кэша
        volatile long long warmup = traverse_with_stride(data, stride);
        (void)warmup;

        // Измерение
        auto start = std::chrono::high_resolution_clock::now();
        long long result = traverse_with_stride(data, stride);
        auto end = std::chrono::high_resolution_clock::now();

        double elapsed_ms = std::chrono::duration<double, std::milli>(end - start).count();
```

```

// Количество загруженных байт: (N / stride) * sizeof(int)
// Но фактически загружаются кэш-линии по 64 байта
double bytes_accessed = static_cast<double>(N / stride) * sizeof(int);
double bandwidth_gbps = (bytes_accessed / 1e9) / (elapsed_ms / 1e3);

std::cout << std::left << std::setw(10) << stride
          << std::setw(20) << std::fixed << std::setprecision(2) << elapsed_ms
          << std::setw(20) << std::fixed << std::setprecision(2) << bandwidth_gbps << "\n";
}

return 0;
}

```

## 2.5. Ассоциативность кэша

Кэш организуется в виде набора **линий** (sets), каждая из которых содержит несколько **путей** (ways). Например, 8-путевый ассоциативный кэш означает, что в каждом наборе может находиться до 8 различных кэш-линий.

- **Прямо-отображаемый кэш (1-way):** Быстрый, но много конфликтных промахов.
- **Полностью ассоциативный кэш:** Минимум конфликтов, но дорогой и медленный.
- **Наборно-ассоциативный (4-way, 8-way):** Компромисс. Современные L1-кэши обычно 8-путевые, L2 — 16-путевые.

### 3. КОГЕРЕНТНОСТЬ КЭШЕЙ: ПРОТОКОЛ MESI, ЛОЖНОЕ СОВМЕСТНОЕ ИСПОЛЬЗОВАНИЕ (15 мин)

#### 3.1. Проблема когерентности

В многоядерных системах каждое ядро имеет собственный кэш (как минимум, уровнями L1 и L2). Когда несколько ядер одновременно работают с одними и теми же данными, возникает проблема **когерентности**: как гарантировать, что все ядра видят актуальные данные?

Без механизма когерентности возможна ситуация, когда ядро 0 записало новое значение переменной *x* в свой кэш, а ядро 1 продолжает читать устаревшее значение из своего кэша.

#### 3.2. Протокол MESI

Наиболее распространённым протоколом когерентности является **MESI** (Modified, Exclusive, Shared, Invalid). Каждая кэш-линия находится в одном из четырёх состояний:

| Состояние            | Описание                                                                             |
|----------------------|--------------------------------------------------------------------------------------|
| <b>Modified (M)</b>  | Линия изменена и присутствует только в данном кэше. Отличается от основной памяти.   |
| <b>Exclusive (E)</b> | Линия не изменена и присутствует только в данном кэше. Совпадает с основной памятью. |
| <b>Shared (S)</b>    | Линия не изменена и присутствует в нескольких кэшах. Совпадает с основной памятью.   |
| <b>Invalid (I)</b>   | Линия недействительна. Должна быть перезагружена при следующем обращении.            |

#### Переходы состояний:

| Действие                            | Из состояния | В состояние                       |
|-------------------------------------|--------------|-----------------------------------|
| Чтение (данных нет в кэше)          | I            | E (если нет в других кэшах) или S |
| Чтение (данные есть в других кэшах) | I            | S                                 |
| Запись                              | M, E         | M                                 |
| Запись                              | S            | M (инвалидация в других кэшах)    |
| Чтение другим ядром                 | E            | S                                 |
| Запись другим ядром                 | M, E, S      | I                                 |

Ссылка: Papamarcos, M. S., & Patel, J. H. (1984). A low-overhead coherence solution for multiprocessors with private cache memories. *ACM SIGARCH Computer Architecture News*, 12(1), 348–354.

#### 3.3. Ложное совместное использование (False Sharing)

**Определение:** Ложное совместное использование — это ситуация, при которой два или более потоков на разных ядрах обращаются к **разным переменным**, но эти переменные расположены в **одной кэш-линии** (64 байта).

**Механизм:** 1. Поток 0 записывает значение в переменную counters[0]. Кэш-линия переходит в состояние Modified в кэше ядра 0. 2. Поток 1 записывает значение в переменную counters[1], находящуюся в той же кэш-линии. Кэш-линия ядра 1 должна быть загружена, но она недействительна (поскольку ядро 0 владеет ей в состоянии M). 3. Ядро 0 вынуждено записать линию в память или передать её ядру 1. 4. Теперь ядро 1 владеет линией в состоянии M. Если поток 0 снова запишет в counters[0], цикл повторяется.

Этот «пинг-понг» кэш-линий между ядрами приводит к **резкому падению производительности** (в 10–100 раз).

**Способы устранения:** 1. **Выравнивание (alignment):** Размещение каждой переменной в отдельной кэш-линии с помощью alignas(64). 2. **Заполнение (padding):** Добавление неиспользуемых байтов между переменными. 3. **Структура данных:** Переход от массива структур к структуре массивов (SoA вместо AoS).

*Ссылка: Boyd-Wickizer, S., et al. (2010). An analysis of Linux scalability to many cores. Proceedings of OSDI '10, 89–103.*

## 4. NUMA-АРХИТЕКТУРЫ: ОСОБЕННОСТИ ДОСТУПА К ПАМЯТИ

### 4.1. Принцип организации

**NUMA (Non-Uniform Memory Access)** — архитектура с неоднородным доступом к памяти. В многопроцессорных системах каждый процессор (сокет) имеет собственную локальную память. Доступ к локальной памяти быстрый, доступ к удалённой памяти (принадлежащей другому сокету) — медленнее.

В современных серверных процессорах (Intel Xeon, AMD EPYC) NUMA-домены могут существовать даже в пределах одного сокета: процессор разделён на несколько **чиплетов (chiplets)**, каждый со своим контроллером памяти.

### 4.2. Влияние на латентность и пропускную способность

Типичные значения для 2-сокетной системы:

| Тип доступа               | Латентность | Пропускная способность |
|---------------------------|-------------|------------------------|
| Локальная память          | ~80 нс      | ~50 ГБ/с               |
| Удалённая память (1 hop)  | ~140 нс     | ~30 ГБ/с               |
| Удалённая память (2 hops) | ~200 нс     | ~20 ГБ/с               |

Разница в латентности может достигать **1.5–3 раз**.

### 4.3. Привязка потоков к ядрам (Thread Affinity)

Для минимизации задержек необходимо: 1. **Привязывать потоки к ядрам** одного NUMA-домена. 2. **Размещать данные в памяти** того же NUMA-домена, где выполняется поток.

В C++ привязка потоков к ядрам осуществляется через `pthread_setaffinity_np` (POSIX) или `SetThreadAffinityMask` (Windows). Для управления размещением памяти используется `numactl` (Linux) или `mbind / set_mempolicy`.

Пример привязки потоков к ядрам:

*// Фрагмент 5: Привязка потоков к ядрам процессора (Thread Affinity) на Linux*

```
#include <iostream>
#include <thread>
#include <vector>
#include <iomanip>
```

```
#ifdef __linux__
#include <pthread.h>
#include <sched.h>
#include <unistd.h>
#endif
```

```
void set_thread_affinity(int core_id) {
#ifdef __linux__
    cpu_set_t cpuset;
    CPU_ZERO(&cpuset);
```

```

CPU_SET(core_id, &cpuset);
int rc = pthread_setaffinity_np(pthread_self(), sizeof(cpu_set_t), &cpuset);
if (rc != 0) {
    std::cerr << "Ошибка привязки потока к ядру " << core_id << "\n";
}
#else
    // Для Windows: SetThreadAffinityMask(GetCurrentThread(), 1 << core_id);
    std::cerr << "Привязка потоков реализована только для Linux в данном примере.\n";
#endif
}

void thread_function(int thread_id) {
    set_thread_affinity(thread_id);

#ifdef __linux__
    // Проверка: на каком ядре реально выполняется поток
    int current_core = sched_getcpu();
    std::cout << "Поток " << thread_id
        << " привязан к ядру " << current_core << "\n";
#endif
}

int main() {
#ifdef __linux__
    int num_cores = std::thread::hardware_concurrency();
    std::cout << "Доступно ядер: " << num_cores << "\n\n";

    std::vector<std::thread> threads;
    int num_threads = std::min(num_cores, 8); // Не более 8 потоков

    for (int i = 0; i < num_threads; ++i) {
        threads.emplace_back(thread_function, i);
    }

    for (auto& t : threads) {
        t.join();
    }

    std::cout << "\nПривязка потоков к ядрам позволяет:\n";
    std::cout << "1. Минимизировать миграцию потоков между ядрами.\n";
    std::cout << "2. Улучшить локальность данных в кэше.\n";
    std::cout << "3. В NUMA-системах обеспечить доступ к локальной памяти.\n";
#else
    std::cout << "Данный пример работает только на системах с ОС Linux.\n";
#endif

    return 0;
}

```

#### 4.4. Практические рекомендации

- Используйте `numactl --hardware` для получения информации о NUMA-топологии.

- При выделении памяти в многопоточных программах используйте **первое касание (first-touch)**: поток, который первым записывает в выделенную память, получает её в своём локальном домене.
- Избегайте миграции потоков между ядрами разных NUMA-доменов.

## 5. ROOFLINE-МОДЕЛЬ: ТЕОРЕТИЧЕСКИЕ ОСНОВЫ, ПОСТРОЕНИЕ, ПРИМЕРЫ

### 5.1. Идея модели

**Roofline-модель** — аналитический инструмент для определения теоретического предела производительности вычислительного алгоритма на данной аппаратной платформе. Модель позволяет определить, является ли алгоритм **ограниченным вычислениями (compute-bound)** или **ограниченным памятью (memory-bound)**.

Идея: производительность алгоритма ограничена либо **пиковой вычислительной способностью** процессора (в FLOPS), либо **пропускной способностью памяти** (в байтах/с), в зависимости от **интенсивности вычислений** (арифметической интенсивности).

*Ссылка: Williams, S., Waterman, A., & Patterson, D. (2009). Roofline: an insightful visual performance model for multicore architectures. Communications of the ACM, 52(4), 65–76.*

### 5.2. Формулы

Арифметическая интенсивность (Arithmetic Intensity)  $I$ :

$$I = \frac{\text{FLOPs}}{\text{Bytes}}$$

где FLOPs — количество операций с плавающей точкой, Bytes — количество байт, переданных между процессором и памятью.

Достижимая производительность  $P$ :

$$P = \min(\pi, \beta \cdot I)$$

где:  $\pi$  — пиковая вычислительная производительность (в FLOPS),  $\beta$  — пиковая пропускная способность памяти (в байтах/с),  $I$  — арифметическая интенсивность.

**Точка пересечения крыши:**  $I^* = \pi/\beta$ . При  $I < I^*$  алгоритм ограничен памятью; при  $I > I^*$  — ограничен вычислениями.

### 5.3. Пример построения для CPU

Для процессора Intel Core i9-13900K: - Пиковая производительность (1 ядро, AVX2, FMA):  $\pi \approx 200$  GFLOPS (для float). - Пропускная способность памяти (2 канала, DDR5-5600):  $\beta \approx 90$  ГБ/с.

Точка пересечения:  $I^* = 200/90 \approx 2.2$  FLOP/byte.

Это означает, что алгоритм должен выполнять **более 2.2 операций с плавающей точкой на каждый загруженный байт**, чтобы быть ограниченным вычислениями.

### 5.4. Пример построения для GPU

Для NVIDIA RTX 4090: - Пиковая производительность:  $\pi \approx 83$  TFLOPS (FP32). - Пропускная способность памяти:  $\beta \approx 1008$  ГБ/с.

Точка пересечения:  $I^* = 83000/1008 \approx 82$  FLOP/byte.

GPU требует значительно более высокой арифметической интенсивности для достижения пиковой производительности.

### 5.5. Использование модели для анализа

| Алгоритм                     | Арифметическая интенсивность | Тип ограничения |
|------------------------------|------------------------------|-----------------|
| Скалярное произведение       | ~0.25 FLOP/byte              | Memory-bound    |
| Сложение векторов            | ~0.125 FLOP/byte             | Memory-bound    |
| Умножение матриц (блочное)   | ~100–500 FLOP/byte           | Compute-bound   |
| Быстрое преобразование Фурье | ~5 FLOP/byte                 | Переходная зона |

## 6. ПРАКТИЧЕСКИЙ КЕЙС: АНАЛИЗ СКАЛЯРНОГО ПРОИЗВЕДЕНИЯ И ДЕМОНСТРАЦИЯ FALSE SHARING

### 6.1. Часть 1: Анализ скалярного произведения векторов

**Задача:** Вычислить скалярное произведение двух векторов  $a \cdot b = \sum_{i=0}^{n-1} a_i \cdot b_i$  и определить тип ограничения алгоритма с помощью Roofline-модели.

**Арифметическая интенсивность скалярного произведения:** - Операции:  $n$  умножений +  $n$  сложений =  $2n$  FLOPs. - Данные:  $2n$  элементов по 4 байта (для float) =  $8n$  байт. -  $I = 2n/8n = 0.25$  FLOP/byte.

При  $I^* \approx 2.2$  FLOP/byte для CPU, скалярное произведение является **сильно ограниченным памятью** ( $0.25 \ll 2.2$ ). Это означает, что оптимизация вычислений (например, использование FMA-инструкций) не даст существенного прироста; необходимо оптимизировать доступ к памяти.

**Эксперимент:** 1. Реализовать скалярное произведение на C++ для векторов различных размеров ( $10^3$ ,  $10^5$ ,  $10^7$ ,  $10^8$  элементов). 2. Измерить время выполнения с помощью `std::chrono::high_resolution_clock`. 3. Вычислить эффективную производительность в GFLOPS и пропускную способность в ГБ/с. 4. Сравнить с теоретическими пределами из Roofline-модели.

Код реализации:

*// Фрагмент 6: Анализ скалярного произведения векторов и построение Roofline*

```
#include <iostream>
#include <vector>
#include <chrono>
#include <cmath>
#include <iomanip>
#include <random>
#include <algorithm>
```

*// Скалярное произведение двух векторов*

```
double dot_product(const std::vector<float>& a, const std::vector<float>& b) {
    double sum = 0.0;
    for (size_t i = 0; i < a.size(); ++i) {
        sum += static_cast<double>(a[i]) * b[i];
    }
    return sum;
}
```

*// Оценка арифметической интенсивности для скалярного произведения*

*// 2n операций (n умножений + n сложений) / 8n байт (2 массива по 4 байта)*

```
double arithmetic_intensity(size_t n) {
    double flops = 2.0 * n;
    double bytes = 8.0 * n; // 2 вектора * 4 байта (float)
    return flops / bytes;
}
```

*// Расчёт теоретической производительности по Roofline-модели*

*//  $P = \min(\pi, \beta * I)$*

```

double roofline_performance(double peak_flops, double peak_bandwidth, double intensity) {
    return std::min(peak_flops, peak_bandwidth * intensity);
}

int main() {
    // Параметры процессора (пример для Intel Core i9-13900K, 1 ядро, AVX2)
    const double PEAK_FLOPS = 200e9;    // 200 GFLOPS (1 ядро, AVX2, FMA, float)
    const double PEAK_BANDWIDTH = 90e9; // 90 ГБ/с (2 канала, DDR5)
    const double RIDGE_POINT = PEAK_FLOPS / PEAK_BANDWIDTH; // ~2.2 FLOP/byte

    std::cout << "=== Roofline-модель ===\n";
    std::cout << "Пиковая производительность: " << PEAK_FLOPS / 1e9 << " GFLOPS\n";
    std::cout << "Пиковая пропускная способность: " << PEAK_BANDWIDTH / 1e9 << " ГБ/с\n";
    std::cout << "Точка пересечения крыши: " << RIDGE_POINT << " FLOP/byte\n\n";

    // Размеры векторов для тестирования
    const std::vector<size_t> sizes = {
        1'000,      // 1К элементов - помещается в L1
        10'000,     // 10К - помещается в L2
        100'000,    // 100К - помещается в L2
        1'000'000,  // 1М - помещается в L3
        10'000'000, // 10М - частично в L3
        100'000'000 // 100М - только в RAM
    };

    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_real_distribution<float> dis(-1.0f, 1.0f);

    std::cout << std::left << std::setw(12) << "Размер"
        << std::setw(12) << "Время (мс)"
        << std::setw(15) << "GFLOPS"
        << std::setw(15) << "ГБ/с"
        << std::setw(15) << "AI (FLOP/B)"
        << std::setw(20) << "Roofline (GFLOPS)"
        << std::setw(15) << "Тип ограничения" << "\n";
    std::cout << std::string(105, '-') << "\n";

    for (size_t n : sizes) {
        // Создание векторов
        std::vector<float> a(n), b(n);
        for (size_t i = 0; i < n; ++i) {
            a[i] = dis(gen);
            b[i] = dis(gen);
        }

        // Разогрев
        volatile double warmup = dot_product(a, b);
        (void)warmup;

        // Измерение времени (среднее по 5 запускам)
        const int NUM_RUNS = 5;
    }
}

```

```

double total_time = 0.0;
for (int run = 0; run < NUM_RUNS; ++run) {
    auto start = std::chrono::high_resolution_clock::now();
    volatile double result = dot_product(a, b);
    (void)result;
    auto end = std::chrono::high_resolution_clock::now();
    total_time += std::chrono::duration<double, std::milli>(end - start).count();
}
double avg_time_ms = total_time / NUM_RUNS;

// Расчёт производительности
double flops = 2.0 * n;
double bytes = 8.0 * n; // 2 вектора * 4 байта
double gflops = (flops / 1e9) / (avg_time_ms / 1e3);
double bandwidth_gbps = (bytes / 1e9) / (avg_time_ms / 1e3);
double ai = arithmetic_intensity(n);

// Теоретический предел по Roofline
double roofline_gflops = roofline_performance(PEAK_FLOPS, PEAK_BANDWIDTH, ai) / 1e9;

// Определение типа ограничения
std::string bottleneck = (ai < RIDGE_POINT) ? "Memory-bound" : "Compute-bound";

std::cout << std::left << std::setw(12) << n
    << std::setw(12) << std::fixed << std::setprecision(3) << avg_time_ms
    << std::setw(15) << std::fixed << std::setprecision(2) << gflops
    << std::setw(15) << std::fixed << std::setprecision(2) << bandwidth_gbps
    << std::setw(15) << std::fixed << std::setprecision(3) << ai
    << std::setw(20) << std::fixed << std::setprecision(2) << roofline_gflops
    << std::setw(15) << bottleneck << "\n";
}

std::cout << "\nВывод: Скалярное произведение имеет арифметическую интенсивность 0.25 FLOP/byte,\n";
std::cout << "что значительно ниже точки пересечения крыши (" << RIDGE_POINT << " FLOP/byte).\n";
std::cout << "Следовательно, алгоритм является ограниченным памятью (memory-bound).\n";
std::cout << "Оптимизация вычислений (например, FMA) не даст существенного прироста;\n";
std::cout << "необходимо оптимизировать доступ к памяти (предвыборка, векторизация загрузки).\n";

return 0;
}

```

## 6.2. Часть 2: Демонстрация False Sharing

**Задача:** Создать массив из  $N$  счётчиков, каждый из которых обновляется отдельным потоком. Сравнить производительность при размещении счётчиков в одном массиве (false sharing) и при выравнивании каждого счётчика до размера кэш-линии.

**Вариант 1: Без выравнивания (с ложным совместным использованием):**

Массив `int counters[N]` размещён в памяти последовательно. При  $N = 8$  потоков и размере кэш-линии 64 байта, в одной линии размещается 16 элементов `int`. Каждый поток обновляет свой счётчик, но кэш-линия «мигрирует» между ядрами.

*// Фрагмент 3: Демонстрация ложного совместного использования (без выравнивания)*

```
#include <iostream>
#include <vector>
#include <thread>
#include <chrono>
#include <iomanip>
```

*// Массив счётчиков без выравнивания - все в одной или нескольких кэш-линиях*

```
struct CountersNoPadding {
    std::vector<long long> counters;
    CountersNoPadding(int n) : counters(n, 0) {}
};

void increment_no_padding(CountersNoPadding& data, int thread_id, int num_iterations) {
    for (int i = 0; i < num_iterations; ++i) {
        data.counters[thread_id]++;
    }
}
```

```
int main() {
    const int NUM_THREADS = 8;
    const int ITERATIONS = 100'000'000; // 100 миллионов инкрементов на поток
```

```
    CountersNoPadding data(NUM_THREADS);
```

```
    std::cout << "Вариант БЕЗ выравнивания (ложное совместное использование)\n";
    std::cout << "Число потоков: " << NUM_THREADS << "\n";
    std::cout << "Число итераций на поток: " << ITERATIONS << "\n\n";
```

```
    auto start = std::chrono::high_resolution_clock::now();
```

```
    std::vector<std::thread> threads;
    threads.reserve(NUM_THREADS);
    for (int i = 0; i < NUM_THREADS; ++i) {
        threads.emplace_back(increment_no_padding, std::ref(data), i, ITERATIONS);
    }
```

```
    for (auto& t : threads) {
        t.join();
    }
```

```
    auto end = std::chrono::high_resolution_clock::now();
    double elapsed_ms = std::chrono::duration<double, std::milli>(end - start).count();
```

```
    std::cout << "Время выполнения: " << std::fixed << std::setprecision(2)
        << elapsed_ms << " мс\n";
```

*// Проверка корректности*

```

    long long total = 0;
    for (int i = 0; i < NUM_THREADS; ++i) {
        total += data.counters[i];
    }
    std::cout << "Суммарное значение: " << total
        << " (ожидается: " << static_cast<long long>(NUM_THREADS) * ITERATIONS << ")\\n";

    return 0;
}

```

## Вариант 2: С выравниванием (без ложного совместного использования):

Каждый счётчик выровнен до 64 байт с помощью alignas(64) или дополнен заполняющими байтами (padding). Теперь каждый поток работает с отдельной кэш-линией.

*// Фрагмент 4: Демонстрация ложного совместного использования (С выравниванием)*

```

#include <iostream>
#include <vector>
#include <thread>
#include <chrono>
#include <iomanip>
#include <new> // для std::hardware_destructive_interference_size

```

*// Каждый счётчик выровнен до размера кэш-линии (64 байта)*

*// Это предотвращает ложное совместное использование*

```

struct alignas(64) PaddedCounter {
    long long value = 0;
};

struct CountersWithPadding {
    std::vector<PaddedCounter> counters;
    CountersWithPadding(int n) : counters(n) {}
};

```

```

void increment_with_padding(CountersWithPadding& data, int thread_id, int num_iterations) {
    for (int i = 0; i < num_iterations; ++i) {
        data.counters[thread_id].value++;
    }
}

```

```

int main() {
    const int NUM_THREADS = 8;
    const int ITERATIONS = 100'000'000; // 100 миллионов инкрементов на поток

```

```

    CountersWithPadding data(NUM_THREADS);

```

```

    std::cout << "Вариант С выравниванием (без ложного совместного использования)\\n";
    std::cout << "Число потоков: " << NUM_THREADS << "\\n";
    std::cout << "Число итераций на поток: " << ITERATIONS << "\\n\\n";

```

```

    auto start = std::chrono::high_resolution_clock::now();

```

```

    std::vector<std::thread> threads;

```

```

threads.reserve(NUM_THREADS);
for (int i = 0; i < NUM_THREADS; ++i) {
    threads.emplace_back(increment_with_padding, std::ref(data), i, ITERATIONS);
}

for (auto& t : threads) {
    t.join();
}

auto end = std::chrono::high_resolution_clock::now();
double elapsed_ms = std::chrono::duration<double, std::milli>(end - start).count();

std::cout << "Время выполнения: " << std::fixed << std::setprecision(2)
    << elapsed_ms << " мс\n";

// Проверка корректности
long long total = 0;
for (int i = 0; i < NUM_THREADS; ++i) {
    total += data.counters[i].value;
}
std::cout << "Суммарное значение: " << total
    << " (ожидается: " << static_cast<long long>(NUM_THREADS) * ITERATIONS << ")\n";

// Сравнение с вариантом без выравнивания
std::cout << "\nВывод: выравнивание счётчиков до размера кэш-линии (64 байта)\n";
std::cout << "устраняет ложное совместное использование и значительно ускоряет программу.\n";

return 0;
}

```

**Ожидаемый результат:** При 8 потоках разница во времени выполнения может составлять 5–50 раз в пользу варианта с выравниванием.

**Анализ счётчиков производительности:** Для подтверждения ложного совместного использования рекомендуется использовать perf stat:

```
perf stat -e cache-misses,cache-references,L1-dcache-load-misses ./false_sharing
```

## ЗАКЛЮЧЕНИЕ

1. Современные процессоры развиваются по пути увеличения числа ядер, а не частоты. Параллельное программирование — обязательный навык.
2. Закон Амдала ограничивает ускорение при фиксированной задаче; закон Густафсона позволяет масштабировать задачу.
3. Иерархия памяти (регистры  $\rightarrow$  L1  $\rightarrow$  L2  $\rightarrow$  L3  $\rightarrow$  RAM) определяет реальную производительность алгоритмов.
4. Когерентность кэш-линий (протокол MESI) обеспечивает корректность данных в многоядерных системах, но порождает проблему ложного совместного использования.
5. В системах с общей памятью на нескольких сокетах используется NUMA-архитектура; привязка потоков и данных критична для производительности.
6. Roofline-модель позволяет определить, является ли алгоритм ограниченным вычислениями или памятью, и выбрать стратегию оптимизации.

## КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Почему скалярное произведение часто бывает ограничено памятью (memory-bound)?
2. Как увеличение размера кэш-линии влияет на ложное совместное использование?
3. Почему закон Густафсона оптимистичнее закона Амдала?
4. Как привязка потоков к ядрам (thread affinity) влияет на производительность в NUMA-системах?
5. Почему обход массива с шагом 256 элементов значительно медленнее обхода с шагом 1?
6. В каких состояниях протокола MESI может находиться кэш-линия, и какие переходы происходят при записи?
7. Как арифметическая интенсивность алгоритма определяет его положение на Roofline-графике?
8. Почему блочное умножение матриц имеет значительно более высокую арифметическую интенсивность, чем скалярное произведение?

## ЛИТЕРАТУРА

1. Williams, S., Waterman, A., & Patterson, D. (2009). Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4), 65–76.
2. Drepper, U. (2007). What Every Programmer Should Know About Memory. *Red Hat, Inc.*
3. Hennessy, J. L., & Patterson, D. A. (2017). Computer Architecture: A Quantitative Approach (6th ed.). Morgan Kaufmann.
4. Fog, A. (2020). Optimizing software in C++: An optimization guide for Windows, Linux and Mac platforms. Copenhagen University College of Engineering.
5. Intel Corporation. (2023). Intel® 64 and IA-32 Architectures Optimization Reference Manual.
6. Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS '67*, 483–485.
7. Gustafson, J. L. (1988). Reevaluating Amdahl's Law. *Communications of the ACM*, 31(5), 532–533.
8. Wulf, W. A., & McKee, S. A. (1995). Hitting the memory wall: Implications of the obvious. *ACM SIGARCH Computer Architecture News*, 23(1), 21–24.
9. Papamarcos, M. S., & Patel, J. H. (1984). A low-overhead coherence solution for multiprocessors with private cache memories. *ACM SIGARCH Computer Architecture News*, 12(1), 348–354.
10. Boyd-Wickizer, S., et al. (2010). An analysis of Linux scalability to many cores. *Proceedings of OSDI '10*, 89–103.
11. Flynn, M. J. (1966). Very High-Speed Computing Systems. *Proceedings of the IEEE*, 54(12), 1901–1909.